PROGRAMMING FUNDAMENTALS

CS – 101

Instructor: Maria N. Samad

October 21st, 2022

STRINGS

- A sequence of characters
- A string literal uses quotes 'Hello' or "Hello"
- For strings, + means “concatenate”
- When a string contains numbers, it is still a string
  - Can convert numbers in a string into a number using int()

STRINGS

- Example:
  - str1 = "Hello“
    str2 = 'there’
    bob = str1 + str2
    print(bob) → Output?
  - str3 = '123’
    str3 = str3 + 1 → Output?
  - x = int(str3) + 1
    print(x) → Output?

READING & CONVERSION

- Data is read as strings, then it is parsed and converted as the user requires
- This gives us more control over error situations and/or bad user input
- Raw input numbers must be converted from strings

READING & CONVERSION

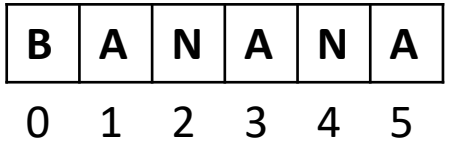
- Example:
  - name = input('Enter: ’) → Output: Enter: Chuck
    print(name) → Output: ?
  - apple = input('Enter: ’) → Output: Enter: 100
    x = apple – 10 → Output: ?
  - x = int(apple) – 10
    print(x) → Output: ?

STRING TRAVERSAL

- We can get at any single character in a string using an index specified in square brackets
- The index value must be an integer and starts at zero
- The index goes to n – 1 where n is the number of characters in a string
- The index value can be an expression that is computed
- Can use negative indexing, in which -1 represents the last letter and then it goes backwards
  - Problem with negative indexing? It won't know when to stop!

STRING TRAVERSAL

- Example:



  - fruit = ‘BANANA’
    letter = fruit[1]
    print(letter) → Output?

  - x = 3
    w = fruit[x – 1]
    print(w) → Output?

INDEX OUT OF RANGE

- You will get a python error if you attempt to index beyond the end of a string
- So be careful when constructing index values and slices
- Example:
  - zot = 'abc’
    print(zot[5]) → Output?

STRING LENGTH

- There is a built-in function ***len*** that gives the length of a string
- Example:
  - fruit = 'banana’
    print(len(fruit)) → Output ?
  - fruit = 'banana’
    x = len(fruit)
    print(x) → Output ?

STRING TRAVERSAL USING LOOPS

- String traversal is the process of going through one character at a time
- Can use any loop to traverse a string
  - ***for*** loop
  - ***while*** loop
- Can traverse using indexing or directly accessing the elements

STRING TRAVERSAL USING LOOPS

- Example for Indexing:
  - fruit = “banana”
    for index in range(len(fruit)):
        letter = fruit[index]
        print(letter)

    OR

  - fruit = “banana”
    index = 0
    while (index < len(fruit)):
        letter = fruit[index]
        print(letter)
        index = index + 1

STRING TRAVERSAL USING LOOPS

- Example for direct letter access:
  - Can only use ***for*** loops, as they are the only ones that provide this functionality

  - fruit = “banana”
    for letter in fruit:
      print(letter)

STRING TRAVERSAL USING LOOPS

- Example: Write a function called ***backtraverse*** that takes a string as argument and displays the letters backwards one letter per line

```
def backtraverse(s):
    lastindex = len(s) – 1
    for index in range(lastindex, -1, -1):
        print(s[index])
```

- In main program, call the function i.e.

```
backtraverse(“Hello”)
```

STRING TRAVERSAL USING LOOPS

- Example: Given two strings: ***prefix*** and ***suffix***, concatenate each letter of the ***prefix*** with the ***suffix*** string according to English Language rules
  - prefix = “JKLMNOPQ”
    suffix = “ack”

```
for letter in prefix:
  if (letter == ‘O’ or letter == ‘Q’):
        print(letter + ‘u’ + suffix)
  else:
        print(letter + suffix)
```

STRING SLICING

- A segment of a string is called a ***slice***
- A ***slice*** may be one or more characters of the string
- Use square brackets with colon operator inside to define the range of slice
- The first number is the starting index in the original string from where the slicing should start
- The second number is one beyond the end of the slice – “up to but not including”
- If the second number is beyond the end of the string, it stops at the end

STRING SLICING

- Example:
  - s = 'Monty Python’
    print(s[0:4]) → Output ?
    print(s[6:7]) → Output ?
    print(s[6:20]) → Output ?

STRING SLICING

- If we leave off the first number or the last number of the slice, it is assumed to be the beginning or end of the string respectively
- If the first index is greater than or equal to second index, it results in empty string
- Example:
  - s = 'Monty Python’
    print (s[:2]) → Output ?
    print (s[8:]) → Output ?
    print (s[:]) → Output ?
    print (s[3:2]) → Output ?

STRINGS ARE IMMUTABLE

- Cannot modify any part of the string like changing one and/or more characters
- This is because they are ***immutable***
- Create a new string with the updated value if needed
- Example:
  - s = ‘Bonty Python’
    s[0] = ‘M’
    print(s) → Output ?
  - new_s = ‘M’ + s[1:]
    print(new_s) → Output ?

STRING CONCATENATION & REPETITION

- When the + operator is applied to strings, it means ***concatenation***
  - Example:
    ```
    a = “Hello”
    b = a + “There”
    print (a + b)        → Output ?
    ```
- When the * operator is applied to strings, it means ***repetition***
  - Example:
    ```
    c = “fun ”
    print (c * 3)        → Output ?
    ```